04. SP1 Core Executor 분석

개요

SP1 Core Executor는 RISC-V 명령어를 해석하고 실행하는 핵심 엔진입니다. 이 컴포넌트는 ELF 바이너리를 로드하여 RISC-V 명령어를 순차적으로 실행하면서, 증명 생성에 필요한 실행 추적(execution trace)을 생성합니다. Executor는 메모리 관리, 시스템 콜 처리, 그리고 다양한 실행 모드를 지원하여 유연하고 효율적인 프로그램 실행 환경을 제공합니다.

핵심 아키텍처

1. Executor 구조체

```
pub struct Executor<'a> {
/// 실행할 프로그램
pub program: Arc<Program>,
/// 실행 상태 (레지스터, 메모리, PC 등)
pub state: ExecutionState,
/// 실행 보고서
pub report: ExecutionReport,
/// 실행 모드
pub executor_mode: ExecutorMode,
/// 메모리 접근 기록
pub memory_accesses: MemoryAccessRecord,
```

```
/// 시스템 콜 매핑
pub syscall_map: HashMap<SyscallCode, Arc<dyn Syscall>>,
/// 실행 기록 (증명 생성용)
pub record: Box<ExecutionRecord>,
// ... 기타 필드들
}
```

주요 특징:

- **프로그램 관리**: Arc를 통한 효율적인 프로그램 공유
- **상태 추적**: 레지스터, 메모리, PC 등 모든 실행 상태 관리
- **실행 기록**: 증명 생성을 위한 상세한 실행 추적 수집
- **시스템 콜**: 확장 가능한 시스템 콜 인터페이스

2. 실행 모드 (ExecutorMode)

```
pub enum ExecutorMode {
/// 추적 없는 단순 실행
Simple,
/// 메모리 체크포인트를 포함한 실행
Checkpoint,
/// 완전한 이벤트 추적을 포함한 실행
Trace,
/// 형태 수집을 위한 크기 제한이 있는 추적
ShapeCollection,
```

```
}
```

각 모드의 특징:

- **Simple**: 최소한의 오버헤드로 빠른 실행
- **Checkpoint**: 메모리 상태 저장으로 중간 복구 가능
- **Trace**: 증명 생성을 위한 완전한 실행 추적
- **ShapeCollection**: 회로 최적화를 위한 형태 정보 수집

RISC-V 명령어 실행 엔진

1. 명령어 구조

```
pub struct Instruction {
/// 실행할 연산
pub opcode: Opcode,
/// 첫 번째 피연산자 (목적지 레지스터)
pub op_a: u8,
/// 두 번째 피연산자
pub op_b: u32,
/// 세 번째 피연산자
pub op_c: u32,
/// 두 번째 피연산자가 즉시값인지 여부
pub imm_b: bool,
/// 세 번째 피연산자가 즉시값인지 여부
```

```
pub imm_c: bool,
}
```

명령어 분류:

- **ALU 명령어**: ADD, SUB, XOR, OR, AND, SLL, SRL, SRA, SLT, SLTU
- **곱셈/나눗셈**: MUL, MULH, MULHU, MULHSU, DIV, DIVU, REM, REMU
- **메모리 접근**: LB, LH, LW, LBU, LHU, SB, SH, SW
- **분기/점프**: BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL, JALR
- **시스템**: ECALL, EBREAK, AUIPC

2. 명령어 실행 사이클

```
impl<'a> Executor<'a> {
/// 단일 명령어 실행
pub fn execute_instruction(&mut self, instruction: &Instruction) -> Result<()
// 1. 피연산자 읽기
let op_a_val = self.register(Register::from(instruction.op_a));
let op_b_val = if instruction.imm_b {
instruction.op_b
} else {
self.register(Register::from(instruction.op_b as u8))
};
let op_c_val = if instruction.imm_c {
instruction.op_c
```

```
} else {
self.register(Register::from(instruction.op_c as u8))
};
// 2. 연산 실행
let result = match instruction.opcode {
Opcode::ADD => op_b_val.wrapping_add(op_c_val),
Opcode::SUB => op_b_val.wrapping_sub(op_c_val),
// ... 기타 연산들
};
// 3. 결과 저장
self.set_register(Register::from(instruction.op_a), result);
// 4. PC 업데이트
self.state.pc += DEFAULT_PC_INC;
Ok(())
}
}
```

메모리 관리 시스템

1. 메모리 구조

```
pub struct Memory<T: Copy> {
/// 32개의 RISC-V 레지스터
```

```
pub registers: Registers<T>,
/// 페이지 테이블 기반 메인 메모리
pub page_table: PagedMemory<T>,
}
```

2. 페이지 기반 메모리 관리

```
pub struct PagedMemory<V: Copy> {
/// 내부 페이지 테이블
pub page_table: Vec<NewPage<V>>,
/// 페이지 인덱스
pub index: Vec<u16>,
}

const LOG_PAGE_LEN: usize = 14; // 16KB 페이지
const PAGE_LEN: usize = 1 << LOG_PAGE_LEN;
const MAX_PAGE_COUNT: usize = ((1 << 31) - (1 << 27)) / 4 / PAGE_LEN + 1;
```

메모리 최적화 특징:

- **지연 할당**: 실제 접근 시에만 페이지 할당
- **압축 주소**: 4바이트 정렬을 활용한 주소 공간 압축
- **효율적인 인덱싱**: 상위/하위 비트 분리를 통한 빠른 접근

3. 메모리 접근 추적

```rust
pub struct MemoryRecord {
/// 메모리 값
pub value: u32,
/// 마지막 접근 샤드
pub shard: u32,
/// 마지막 접근 타임스탬프
pub timestamp: u32,
}


impl<'a> Executor<'a> {
/// 메모리 읽기 및 접근 기록 생성
pub fn mr(&mut self, addr: u32, shard: u32, timestamp: u32) -> MemoryReadReco
// 주소 유효성 검사
if addr % 4 != 0 || addr <= Register::X31 as u32 || addr >= BABYBEAR_PRIME {
panic!("Invalid memory access: addr={addr}");
}
// 메모리 엔트리 가져오기
let entry = self.state.memory.page_table.entry(addr);
// 이전 기록 저장
let prev_record = match entry {
Entry::Occupied(entry) => *entry.get(),
Entry::Vacant(entry) => {
```

```
let  initial_value = self.state.uninitialized_memory

.page_table.get(addr).unwrap_or(&0);

entry.insert(MemoryRecord {

value: *initial_value,

shard: 0,

timestamp: 0

})

}

};

// 현재 접근 기록 업데이트

let  record = entry.into_mut();

record.shard = shard;

record.timestamp = timestamp;

// 메모리 읽기 기록 생성

MemoryReadRecord::new(

record.value,

record.shard,

record.timestamp,

prev_record.shard,

prev_record.timestamp,

)

}
```

```
}
```

시스템 콜 처리

1. 시스템 콜 인터페이스

```
pub trait Syscall: Send + Sync {
/// 시스템 콜 실행
fn execute(
&self,
ctx: &mut SyscallContext,
syscall_code: SyscallCode,
arg1: u32,
arg2: u32,
) -> Option<u32>;
/// 추가 사이클 수
fn num_extra_cycles(&self) -> u32 {
0
}
}
```

2. 주요 시스템 콜들

```
pub enum SyscallCode {
```

```
/// 프로그램 종료
HALT = 0x00_00_00_00,
/// SHA256 관련
SHA_EXTEND = 0x00_00_30_26,
SHA_COMPRESS = 0x00_00_01_05,
/// 타원곡선 암호화
ED_ADD = 0x00_01_01_28,
ED_DECOMPRESS = 0x00_01_01_29,
SECP256K1_ADD = 0x00_01_01_14,
SECP256K1_DOUBLE = 0x00_01_01_15,
/// 메모리 및 I/O
WRITE = 0x00_00_00_02,
COMMIT = 0x00_00_00_10,
/// 제약 없는 실행
ENTER_UNCONSTRAINED = 0x00_00_00_F0,
EXIT_UNCONSTRAINED = 0x00_00_00_F1,
// ... 기타 시스템 콜들
}
```

3. 시스템 콜 실행 과정

```
impl<'a> Executor<'a> {
/// ECALL 명령어 처리
```

```
pub fn execute_ecall(&mut self) -> Result<()> {
// 시스템 콜 코드 읽기 (a7 레지스터)
let syscall_code = SyscallCode::from(self.register(Register::X17));
// 인자 읽기 (a0, a1 레지스터)
let arg1 = self.register(Register::X10);
let arg2 = self.register(Register::X11);
// 시스템 콜 찾기 및 실행
if let Some(syscall) = self.syscall_map.get(&syscall_code) {
let mut ctx = SyscallContext::new(self);
let result = syscall.execute(&mut ctx, syscall_code, arg1, arg2);
// 결과를 a0 레지스터에 저장
if let Some(result) = result {
self.set_register(Register::X10, result);
}
// 추가 사이클 처리
let extra_cycles = syscall.num_extra_cycles();
self.state.clk += extra_cycles;
} else {
return Err(ExecutionError::UnsupportedSyscall(syscall_code as u32));
}
Ok(())
}
```

```
}
```

실행 추적 생성

1. 실행 기록 구조

```rust
pub struct ExecutionRecord {
/// 프로그램 참조
pub program: Arc<Program>,
/// CPU 이벤트들
pub cpu_events: Vec<CpuEvent>,
/// ALU 이벤트들
pub alu_events: Vec<AluEvent>,
/// 메모리 이벤트들
pub memory_initialize_events: Vec<MemoryInitializeFinalizeEvent>,
pub memory_finalize_events: Vec<MemoryInitializeFinalizeEvent>,
/// 분기/점프 이벤트들
pub branch_events: Vec<BranchEvent>,
pub jump_events: Vec<JumpEvent>,
/// 시스템 콜 이벤트들
pub syscall_events: Vec<SyscallEvent>,
// ... 기타 이벤트들
}
```

2. 이벤트 생성 과정

```
impl<'a> Executor<'a> {
/// CPU 이벤트 생성
pub fn emit_cpu_event(&mut self, instruction: &Instruction) {
if self.executor_mode == ExecutorMode::Trace {
let event = CpuEvent {
clk: self.state.clk,
pc: self.state.pc,
instruction: *instruction,
a: self.register(Register::from(instruction.op_a)),
b: if instruction.imm_b {
instruction.op_b
} else {
self.register(Register::from(instruction.op_b as u8))
},
c: if instruction.imm_c {
instruction.op_c
} else {
self.register(Register::from(instruction.op_c as u8))
},
};
self.record.cpu_events.push(event);
```

```
}
}
/// 메모리 이벤트 생성
pub fn emit_memory_event(&mut self, addr: u32, value: u32, is_read: bool) {
if self.executor_mode == ExecutorMode::Trace {
let event = MemInstrEvent {
clk: self.state.clk,
addr,
value,
is_read,
shard: self.state.current_shard,
timestamp: self.timestamp(&MemoryAccessPosition::Memory),
};
self.record.memory_events.push(event);
}
}
}
```

3. 샤드 분할 및 관리

```
impl<'a> Executor<'a> {
/// 새로운 샤드 시작
pub fn start_new_shard(&mut self) {
```

```
// 현재 기록을 완료된 기록 목록에 추가
let mut new_record = ExecutionRecord::new(self.program.clone());
std::mem::swap(&mut self.record, &mut new_record);
self.records.push(new_record);
// 샤드 번호 증가
self.state.current_shard += 1;
// 클럭 리셋
self.state.clk = 0;
// 메모리 체크포인트 초기화
self.memory_checkpoint.clear();
self.uninitialized_memory_checkpoint.clear();
}
/// 샤드 크기 확인
pub fn should_split_shard(&self) -> bool {
self.state.clk >= self.shard_size
}
}
```

성능 최적화 기법

1. 제약 없는 실행 모드

```
impl<'a> Executor<'a> {
```

```
/// 제약 없는 모드 진입
pub fn enter_unconstrained(&mut self) {
// 현재 상태 백업
self.unconstrained_state = Box::new(ForkState {
clk: self.state.clk,
pc: self.state.pc,
memory_diff: HashMap::new(),
register_diff: HashMap::new(),
});
self.unconstrained = true;
}
/// 제약 없는 모드 종료
pub fn exit_unconstrained(&mut self) {
// 상태 복원 (메모리 변경사항만 유지)
self.state.clk = self.unconstrained_state.clk;
self.state.pc = self.unconstrained_state.pc;
// 메모리 변경사항 롤백
for (addr, original_value) in &self.unconstrained_state.memory_diff {
match original_value {
Some(record) => {
self.state.memory.insert(*addr, *record);
}
None => {
```

```
self.state.memory.remove(*addr);
}
}
}
self.unconstrained = false;
}
}
```

2. 메모리 접근 최적화

```
impl<'a> Executor<'a> {
/// 로컬 메모리 접근 최적화
pub fn optimize_local_memory_access(&mut self) {
// 자주 접근되는 메모리 주소들을 로컬 캐시에 저장
let mut local_access_count = HashMap::new();
for (addr, _) in &self.local_memory_access {
*local_access_count.entry(*addr).or_insert(0) += 1;
}
// 접근 빈도가 높은 주소들을 우선 처리
let mut sorted_addresses: Vec<_> = local_access_count.into_iter().collect();
sorted_addresses.sort_by(|a, b| b.1.cmp(&a.1));
// 상위 N개 주소에 대해 최적화된 접근 패턴 적용
for (addr, _) in sorted_addresses.iter().take(NUM_LOCAL_MEMORY_ENTRIES_PER_ROW_
```

```
if let Some(event) = self.local_memory_access.get(addr) {
// 최적화된 메모리 이벤트 생성
self.emit_optimized_memory_event(event);
}
}
}
}
```

3. 실행 추정 및 조기 종료

```
impl<'a> Executor<'a> {
/// LDE 크기 추정 및 조기 종료
pub fn check_lde_size_threshold(&self) -> Result<()> {
if self.lde_size_check && self.state.clk % self.shape_check_frequency == 0 {
let estimated_lde_size = estimate_riscv_lde_size(&self.local_counts);
if estimated_lde_size > self.lde_size_threshold {
return Err(ExecutionError::ExceededCycleLimit(self.lde_size_threshold));
}
}
Ok(())
}
/// 사이클 제한 확인
pub fn check_cycle_limit(&self) -> Result<()> {
```

```
if let Some(max_cycles) = self.max_cycles {
if self.state.global_clk >= max_cycles {
return Err(ExecutionError::ExceededCycleLimit(max_cycles));
}
}
Ok(())
}
}
```

디버깅 및 프로파일링

1. 실행 보고서

```
pub struct ExecutionReport {
/// 총 명령어 수
pub total_instruction_count: u64,
/// 각 명령어 타입별 실행 횟수
pub opcode_counts: HashMap<Opcode, u64>,
/// 시스템 콜 실행 횟수
pub syscall_counts: HashMap<SyscallCode, u64>,
/// 메모리 사용량
pub memory_usage: MemoryUsage,
/// 실행 시간 정보
```

```
pub execution_time: Duration,
/// 가스 사용량 (선택적)
pub gas_usage: Option<u64>,
}


impl ExecutionReport {
/// 보고서 출력
pub fn print_summary(&self) {
println!("=== SP1 Execution Report ===");
println!("Total Instructions: {}", self.total_instruction_count);
println!("Execution Time: {:?}", self.execution_time);
if let Some(gas) = self.gas_usage {
println!("Gas Usage: {}", gas);
}
println!("\nTop Instructions:");
let mut sorted_opcodes: Vec<_> = self.opcode_counts.iter().collect();
sorted_opcodes.sort_by(|a, b| b.1.cmp(a.1));
for (opcode, count) in sorted_opcodes.iter().take(10) {
println!(" {}: {}", opcode.mnemonic(), count);
}
}
}
```

2. 프로파일링 지원

```
#[cfg(feature = "profiling")]
impl<'a> Executor<'a> {
/// 프로파일러 설정
pub fn setup_profiler(&mut self, elf_bytes: &[u8]) {
if let Ok(trace_file) = std::env::var("TRACE_FILE") {
let sample_rate = std::env::var("TRACE_SAMPLE_RATE")
.ok()
.and_then(|rate| rate.parse::<u32>().ok())
.unwrap_or(1);
let profiler = Profiler::new(elf_bytes, sample_rate as u64)
.expect("Failed to create profiler");
let trace_buf = BufWriter::new(
File::create(trace_file).expect("Failed to create trace file")
);
self.profiler = Some((profiler, trace_buf));
}
}
/// 프로파일링 샘플 수집
pub fn collect_profile_sample(&mut self) {
if let Some((profiler, trace_buf)) = &mut self.profiler {
if self.state.clk % profiler.sample_rate == 0 {
```

```
profiler.sample(self.state.pc, self.state.clk, trace_buf);

}

}

}

}
```

요약

SP1 Core Executor는 RISC-V 명령어 실행의 핵심을 담당하는 정교한 시스템입니다. 페이지 기반 메모리 관리, 확장 가능한 시스템 콜 인터페이스, 그리고 다양한 실행 모드를 통해 효율적이고 유연한 프로그램 실행 환경을 제공합니다. 특히 증명 생성을 위한 상세한 실행 추적 수집과 성능 최적화 기법들이 SP1의 고성능 영지식 증명 생성을 가능하게 합니다.